DIKE: An Enterprise-Grade Carpooling and Ridesharing Platform Engineered with Spring Boot Modular Monolith, Next.js, and React Native Expo

Comprehensive End-to-End System Report on Web, Mobile, and Backend Architecture

Phan Chi Thong (QE190072), Ho Le Quoc Khanh (QE190009),
Nguyen Cong Nhat (QE190017), Nguyen Ai Linh (QE190107),
Tran Nguyen Bao Long (QE190010)
Student Developers, Class: SEK19B.Net
Department of Software Engineering
FPT University Quy Nhon Campus, Binh Dinh, Vietnam

Huynh Vang
Project Advisor, Lecturer
Department of Software Engineering
FPT University Quy Nhon Campus, Binh Dinh, Vietnam
Email: vangh@fe.edu.vn

***Abstract*—Daily commuter transport between suburban residential areas and regional university hubs faces severe bottlenecks, leading to traffic congestion, heightened carbon footprints, and escalating personal fuel expenses. Although carpooling presents an ecologically and economically viable remedy, conventional ad-hoc arrangements across social media groups suffer from fragmented communication, unreliable commitments, personal safety concerns, and the total absence of real-time transit state tracking. This paper presents the comprehensive engineering and empirical evaluation of Dike — an enterprise-grade, greenfield ridesharing and carpooling ecosystem designed for a controlled 500-user production pilot. The platform adopts a monorepo topology consisting of: (1) an architectural Backend built with Java 21 LTS and Spring Boot 4.1, structured as a Modular Monolith adhering to Clean Architecture principles, verified continuously by Spring Modulith and ArchUnit, with concurrency-safe state transitions guaranteed via MongoDB Replica Set distributed transactions and the Transactional Outbox Pattern; (2) a responsive Web Client engineered with Next.js 16 (App Router) and React 19, incorporating a secure Same-Origin Authentication Proxy and an ultra-modern Tri-Island Floating Glass design system with Three.js 3D visualizations; and (3) a cross-platform Mobile Application developed with React Native 0.86 and Expo SDK 57, leveraging file-based Expo Router, hardware-backed SecureStore token encryption, and native background location tracking. Rigorous multi-tiered automated verification (ArchUnit, Testcontainers, Vitest, and Playwright E2E) demonstrates absolute data consistency (zero overbooking), seamless real-time message delivery, and robust defense-in-depth security.**

***Index Terms*—Carpooling, Ridesharing, Spring Boot, Modular Monolith, Clean Architecture, Next.js, React Native Expo, Concurrency Control, Transactional Outbox, WebSocket/STOMP.**

I. Introduction

Urbanization and the spatial dispersion of university campuses (such as FPT University Quy Nhon Campus) have generated substantial recurring daily transit demand. Thousands of university students, academic staff, and office employees travel along overlapping trajectories during identical rush-hour intervals. While ride-sharing offers an intuitive solution, informal coordination via online messaging channels (Facebook Groups, Zalo) introduces critical vulnerabilities: (1) unindexed and volatile trip posts; (2) lack of binding reservation mechanisms, causing frequent last-minute cancellations; (3) absence of driver/passenger identity verification; and (4) inadvertent exposure of private personally identifiable information (PII) such as personal phone numbers and home addresses.

The **Dike** project was conceived to address these limitations through a dedicated, full-stack digital platform. The system orchestrates the entire transit lifecycle: recurring weekly schedule management, automated travel intent materialization, spatial route corridor matching, bilateral ride proposals, atomic seat reservations, real-time messaging, secure one-time boarding code verification, fair fuel-cost contribution settlements, and double-blind post-trip reviews.

The primary engineering contributions of this work include:

- **Pragmatic Architectural Paradigm:** Rejection of both brittle unorganized monoliths and prematurely distributed microservices in favor of a *Spring Boot Modular Monolith*. This enforces strict domain boundaries (Bounded Contexts) while preserving ACID transaction boundaries and minimizing operational DevOps overhead.
- **Provable Concurrency Control:** Elimination of seat overbooking race conditions during simultaneous reservation acceptances via MongoDB Replica Set multi-document transactions and atomic conditional writes.
- **Contract-Driven Development:** Single-source-of-truth API design governed by OpenAPI 3.1 specifications, with

an automated generation pipeline providing type-safe TypeScript clients across both Web and Mobile platforms.
- **Modern Cross-Platform User Experience:** Cohesive implementation of the Urban Mint and Floating Glass design systems across Next.js 16 and Expo SDK 57, supplemented by interactive 3D WebGL route previews.

II. System Architecture Overview

A. High-Level Topology

The Dike ecosystem maintains strict architectural boundaries across the Presentation Layer, Application Core, and Persistence/Integration Services, as illustrated in Fig. 1.

B. Single Source of Truth & State Classification

To maintain state consistency under concurrent network conditions, Dike classifies data into four distinct tiers:

1) **Durable Business State:** Persisted exclusively in the MongoDB Replica Set. All contractual commitments (user records, financial contributions, trip bookings, messages) must achieve durable commit status in MongoDB prior to triggering side effects.
2) **Ephemeral Volatile State:** Maintained in-memory within Redis, including opaque user sessions, one-time WebSocket connection tickets (30-second TTL), rate-limiting counters, Goong route caches (10-minute TTL), and real-time GPS locations (120-second TTL).
3) **Binary Blob Storage:** Hosted in Cloudflare R2 (or local MinIO), partitioned into Public Buckets (user avatars) and Private Buckets (driver licenses, vehicle registrations). The database maintains metadata and ownership references only.
4) **Client Presentation State:** Managed locally by TanStack React Query. Real-time WebSocket messages are treated as hints/signals; clients retain the capability to re-fetch canonical state via REST APIs upon reconnection.

C. Monorepo Organization & Technology Stack

The repository is structured as an integrated monorepo combining `pnpm` workspaces and the Maven Wrapper:

Table I
Monorepo Module and Technology Distribution

Workspace Path	Technology / Framework	Core Responsibility
`apps/backend`	Java 21, Spring Boot 4.1.1, Modulith	Modular Monolith API & Worker
`apps/web`	Next.js 16.3, React 19, TypeScript	Web Portal & Auth Proxy
`apps/mobile`	React Native 0.86, Expo 57, Router	Native Mobile Client (iOS/Android)
`api-client-ts`	Generated TypeScript Client	Shared Network SDK
`design-tokens`	CSS Design Tokens	Urban Mint Design Tokens
`packages/web-ui`	React Component Library	Shared UI Primitives
`infra/`	Docker Compose, Render Blueprints	Containerized Local Infra

III. Backend Design and Implementation

A. Spring Boot Modular Monolith Architecture

To circumvent the distributed transaction overhead of microservices while avoiding the structural decay of monolithic codebases, the backend is architected as a **Modular Monolith** under the root package `com.dike`, organized into 7 autonomous Bounded Contexts.

Layering within each module strictly follows Clean Architecture:

$$\text{API Layer} \xrightarrow{\text{calls}} \text{Application Layer} \xrightarrow{\text{uses}} \text{Domain Model} \xleftarrow{\text{implements}} \text{Infrastructure Layer} \tag{1}$$

- **API Layer:** Houses REST Controllers and Request/Response DTOs. It validates input formatting and delegates execution immediately to application use cases without accessing repositories.
- **Application Layer:** Coordinates use cases, enforces transaction boundaries (`@Transactional`), verifies actor permissions, and publishes transactional outbox events.
- **Domain Layer:** Contains pure domain entities, value objects, and finite state machines. It remains completely free from dependencies on Spring Data, MongoDB, or external vendor SDKs.
- **Infrastructure Layer:** Implements domain ports: Spring Data MongoDB repositories, Redis gateways, Goong routing adapters, and S3-compatible storage clients.

Architectural integrity is continuously validated via **ArchUnit** and **Spring Modulith**. Listing 1 depicts the automated architectural boundary test:

Listing 1. Automated Architectural Verification via ArchUnit

```
@Test
void controllersMustNotDependOnRepositories() {
  classes().that().haveSimpleNameEndingWith("Controller")
    .should().onlyDependOnClassesThat()
    .resideOutsideOfPackages("..infrastructure..", "..
        repository..")
    .check(classes);
}
@Test
void domainMustNotDependOnFrameworks() {
  noClasses().that().resideInAPackage("..domain..")
    .should().dependOnClassesThat()
    .resideInAnyPackage("org.springframework.web..",
                        "org.springframework.data..",
                        "software.amazon.awssdk..")
    .check(classes);
}
```

B. Bounded Contexts Breakdown

1) Module `identity`: Governs user identity, credentials, and access control:

- **Google OAuth2 / OIDC:** Implements Authorization Code Flow with PKCE, validating cryptographic `state`, `nonce`, and OpenID Connect tokens.
- **Opaque Session Management:** Produces cryptographically random 256-bit session tokens stored in Redis under `dike:session`. Web clients utilize `HttpOnly`,

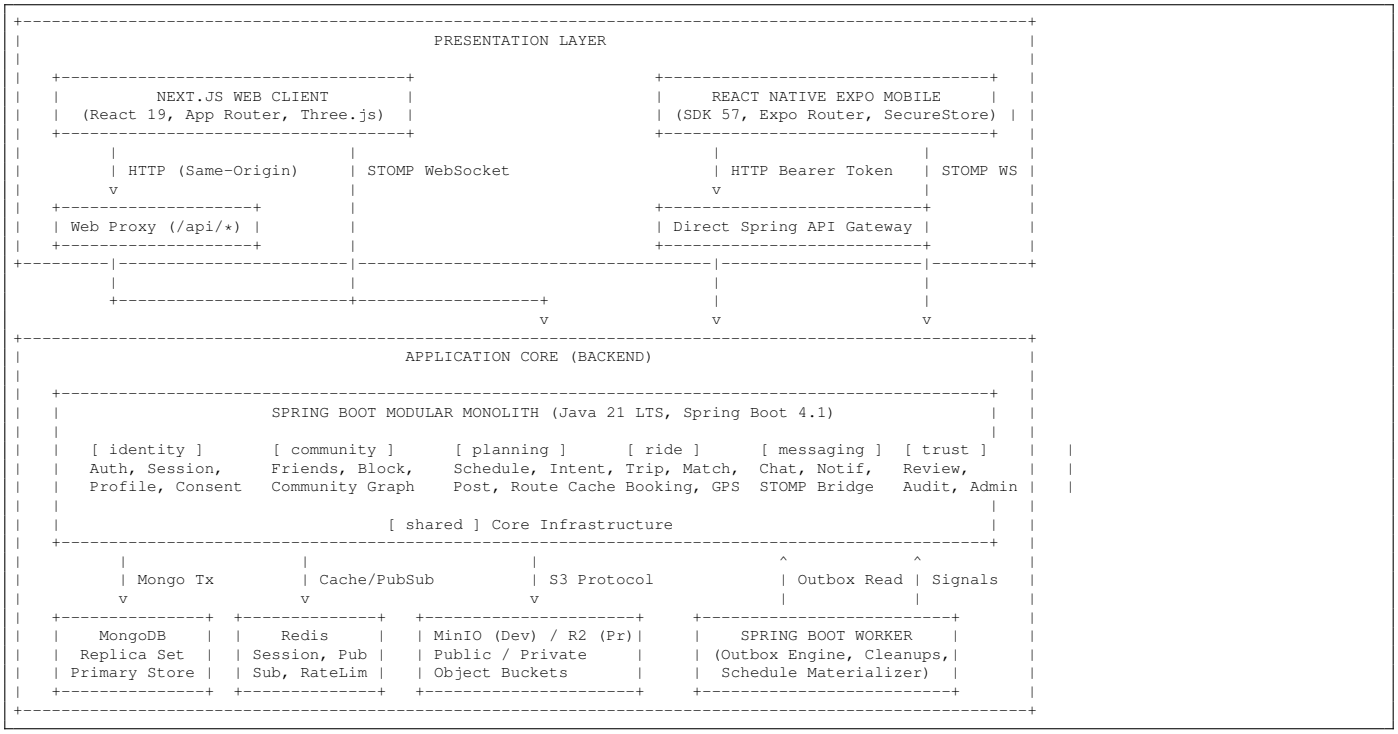


Figure 1. Dike End-to-End System Architecture (Backend Modular Monolith, Web Proxy, and Native Mobile Client).

`Secure`, `SameSite=Lax` cookies; mobile clients leverage Bearer headers.

- **Phone OTP Verification:** Normalizes phone numbers to international E.164 formatting. 6-digit OTPs are hashed using HMAC-SHA256 with secret keys prior to storage. Rate limiting enforces a maximum of 5 attempts per OTP, 5-minute validity, and a 60-second cooldown.
- **Dynamic Authorization:** Defines hierarchical authorities (`MEMBER`, `VERIFIED_MEMBER`, `APPROVED_DRIVER`, `MODERATOR`, `ADMIN`), evaluated dynamically by resource policy services.

2) Module `community`: Manages secure bounded social relationships:

- **Institutional Communities:** Represents closed organizations (universities, corporate parks) requiring verified membership approval.
- **Normalized Friendship Graph:** Normalizes user pairs into canonical representations $\min(U_A, U_B) : \max(U_A, U_B)$ to prevent duplicate edges. Enforces a 7-day cooldown period upon friend request rejection.
- **Bidirectional Blocking:** Carries absolute precedence. Activating a block immediately revokes friendships, dismisses pending proposals, hides mutual profiles/posts, and terminates active matching corridors.

3) Module `planning`: Automates recurring travel demand:

- **Recurring Schedule Rules (`ScheduleRule`):** Configures habitual weekly commuting schedules (e.g., Mondays/Wednesdays/Fridays departing from the dorm to the campus before 07:15). Supports daily overrides via `ScheduleException`.
- **Travel Intent Materialization:** An automated background worker materializes recurring rules into concrete `TravelIntent` records 14 days in advance. A composite unique key on `userId + date + direction` guarantees idempotent execution.
- **Ride Posts (`RidePost`):** Exposes public listings for ride requests and offers across three visibility scopes: `PUBLIC`, `COMMUNITY`, and `FRIENDS`.

4) Module `ride`: Coordinates vehicle supply, route coordination, and reservation state machines:

- **Trip Aggregates (`Trip`):** Managed by drivers, maintaining planned stops (`TripStop`), departure timestamps, capacity, and detour constraints.
- **Bilateral Proposals (`RideProposal`):** Facilitates bidirectional negotiation (passengers requesting seats or drivers offering pickups).
- **Bookings (`Booking`):** Encapsulates mutually binding ride agreements created upon proposal acceptance.
- **One-Time Boarding Code:** Issues a randomized 4-digit secret visible exclusively to the passenger; the driver must submit this code at pickup to transition state to `BOARDED`.

5) Module `messaging`: Coordinates real-time interpersonal and contextual communication:

- Segregates conversations into `DIRECT` (bilateral user chat) and `BOOKING` (scoped strictly to an active ride).
- Implements idempotent message dispatching via client-generated `clientMessageId` tokens.

- Separates incoming communication into verified chats and untrusted *Message Requests*.

6) Module `trust`: Guarantees community reputation and operational moderation:

- **Double-Blind Post-Trip Reviews:** Both parties submit ratings (1–5 stars) and qualitative feedback. Reviews remain completely confidential until both submissions are finalized or a 7-day deadline expires, mitigating retaliatory negative reviews.
- **Incident Reporting & Moderation:** Allows reporting suspicious accounts, ride disputes, or offensive messages.
- **Immutable Audit Log:** Records every administrative intervention (account locks, content takedowns) with UTC timestamps and operator IDs.

7) Module `shared`: Provides cross-cutting utilities: standardized UTC clocks, unified RFC-7807 `ErrorResponse` contracts, Transactional Outbox primitives, and Micrometer observability.

C. Concurrency Control & Race Condition Prevention

The critical challenge in high-concurrency ridesharing lies in preventing **overbooking** when multiple passengers concurrently attempt to accept proposals for the final remaining seat.

Dike resolves this race condition through a synthesis of **MongoDB Multi-Document Distributed Transactions** and **Atomic Conditional Updates**:

Listing 2. Atomic Conditional Seat Decrement

```
@Transactional
public Booking acceptProposal(String proposalId, String
    userId) {
  RideProposal proposal = proposalRepo.findById(proposalId)
      .orElseThrow(() -> new NotFoundException("
          PROPOSAL_NOT_FOUND"));
  proposal.validateCanBeAcceptedBy(userId);

  // Atomic conditional decrement in MongoDB
  Query query = new Query(Criteria.where("_id").is(proposal
      .getTripId())
      .and("remainingSeats").gt(0)
      .and("status").is(TripStatus.OPEN.name())
      .and("version").is(proposal.getTripVersion()));

  Update update = new Update()
      .inc("remainingSeats", -1)
      .inc("version", 1);

  Trip updatedTrip = mongoTemplate.findAndModify(query,
      update,
      FindAndModifyOptions.options().returnNew(true), Trip.
          class);

  if (updatedTrip == null) {
    throw new BusinessRuleException("
        NO_SEATS_OR_TRIP_CHANGED",
        "Trip has no remaining seats or has changed!");
  }

  // Create booking and snapshot agreed conditions
  Booking booking = Booking.createFromProposal(proposal,
      updatedTrip);
  bookingRepo.save(booking);

  proposal.transitionToAccepted();
  proposalRepo.save(proposal);

  // Publish outbox event within the same database
      transaction
  outboxPublisher.publish("booking.created", booking.getId
      (), booking);
  return booking;
}
```

When N simultaneous requests compete for a single seat, exactly one `findAndModify` operation satisfies the conditional predicate `remainingSeats > 0`. The remaining $N - 1$ threads receive a `null` document, triggering an immediate exception and transaction rollback, guaranteeing provable zero-overbooking.

D. Transactional Outbox Pattern & Asynchronous Processing

To maintain eventual consistency across secondary projections and external integrations without degrading REST response latency, Dike implements the **Transactional Outbox Pattern**. Application state changes and corresponding `OutboxEvent` records are committed atomically within the same database transaction.

The background worker lifecycle proceeds as follows:

1) **Atomic Lease Acquisition:** Workers execute atomic `findAndModify` queries to claim batches of `PENDING` events whose lease has expired (`lockedUntil < Instant.now()`), updating `lockedUntil = now() + 30s`.
2) **Idempotent Dispatch:** The worker dispatches events via Redis Pub/Sub and records delivery receipts (`OutboxReceipt`).
3) **Exponential Backoff Retry:** Failed dispatches increment the attempt counter and calculate backoff intervals with jitter:

$$T_{\text{next}} = T_{\text{current}} + T_{\text{base}} \times 2^{\text{attempts}} + \text{jitter} \qquad (2)$$

Events exceeding 8 attempts transition to `FAILED` for administrator review.

E. Real-Time Subsystem: WebSocket/STOMP & Redis Bridge

Dike segregates command ingestion (via REST) from real-time event broadcasting (via WebSocket):

- **One-Time Connection Tickets:** Clients never transmit session cookies or authorization tokens in WebSocket URL query strings. Instead, the client invokes `POST /api/v1/realtime/ticket` to obtain a 256-bit single-use ticket stored in Redis (30-second TTL), consumed atomically during STOMP `CONNECT`.
- **Redis Pub/Sub Multi-Instance Bridge:** Across scaled API deployments, background workers publish event envelopes to shared Redis channels. All API instances receive these broadcasts and route them directly to user-specific STOMP queues: `/user/queue/notifications`, `/user/queue/bookings`.

F. Routing Algorithm & Fair Cost-Sharing Model

Integration with the Goong Maps API provides precise geographical routing:

- **Corridor Matching:** MongoDB `2dsphere` spatial indexes filter candidate trips whose primary trajectory

passes within 2 km of pickup points. Candidates undergo detour analysis: maximum additional distance $\leq$ 3 km and detour delay ≤ 10 minutes.

- **Non-Commercial Cost Contribution:** In compliance with non-commercial ridesharing principles, passenger contributions reflect shared fuel costs:

$$R_{\text{km}} = \frac{P_{\text{fuel}} \times C_{100\text{km}}}{100 \times (N_{\text{seats}} + 1)} \tag{3}$$

$$\text{Contribution} = \text{Round}_{1000}\left(D_{\text{shared}} \times R_{\text{km}}\right) \tag{4}$$

where P_{fuel} is the fuel price (VND/L), $C_{100\text{km}}$ represents standard consumption (2.2 L/100km for motorbikes, 8.0 L/100km for cars), and N_{seats} denotes vehicle capacity.

IV. Web Frontend Architecture and Implementation

A. Next.js 16 App Router & Same-Origin Auth Proxy

The web client is engineered using Next.js 16 and React 19, utilizing the App Router to separate Server Components from interactive Client Components.

A fundamental security innovation is the **Same-Origin Authentication Proxy**: Browser runtimes never interact directly with the backend host. Instead, Next.js maintains an internal reverse proxy under the route namespace `/api/*`. All client requests target the local origin (`http://localhost:3000/api/...`), which the Next.js server forwards internally to the Spring Boot API (`http://localhost:8080/api/...`).

- The session cookie `DIKE_SESSION` is configured with `SameSite=Lax` and `HttpOnly`, eliminating session hijacking vectors via XSS.
- Eliminates reliance on brittle third-party cookies across modern web browsers.
- Validates `Origin` and `X-Requested-With` headers to prevent Cross-Site Request Forgery (CSRF).

B. Tri-Island Floating Glass Design System

The application introduces a luxury **Tri-Island Floating Navbar** based on the Urban Mint design system:

1) **Brand Island (Left):** Houses the Dike logo and styled typography with subtle specular inner glows.
2) **Navigation Island (Center):** A centered floating capsule hosting primary links (Home, Explore, Friends), collapsing dynamically on mobile viewports.
3) **Action Island (Right):** Contains notification badges with unread counters, chat shortcuts, and user profile controls.

The visual appearance is achieved using multi-layered optical CSS:

Listing 3. Optical Liquid Glass CSS Styling

```
background: rgba(255, 255, 255, 0.75);
backdrop-filter: blur(16px) saturate(180%);
border: 1px solid rgba(255, 255, 255, 0.4);
box-shadow: 0 8px 32px 0 rgba(16, 185, 129, 0.08);
```

C. Interactive 3D Graphics & Unified Workflow

- **Three.js 3D Viewport:** The landing page renders a dynamic WebGL canvas featuring interactive geographical models, enhancing user engagement.
- **Smart Root Routing (`app/page.tsx`):** The root route dynamically evaluates session status: unauthenticated visitors view the marketing landing page, while authenticated users transition instantly to the main dashboard (`/home`) using branded skeletons to prevent layout shift.
- **Unified "Offer / Request Ride" Flow (`/rides/new`):** Consolidates passenger and driver creation workflows into a single interface with instant role switching, MapLibre GL route previews, and automated proposal matching.

V. Mobile Application Architecture and Implementation

A. React Native 0.86 & Expo SDK 57 Foundation

The native mobile application is structured under `apps/mobile` using **React Native 0.86.3** and **Expo SDK 57.0.27**:

- **Expo Router v57:** File-based routing maps screen components in `src/app/` directly to native navigation stacks.
- **Continuous Native Generation (CNG):** Avoids brittle checked-in native folders, configuring native Android and iOS parameters entirely via `app.json` and Expo Config Plugins.

B. Native Authentication & Hardware-Backed Security

In contrast to the Web's cookie proxy model, mobile clients communicate directly with Spring Boot via Bearer Tokens:

1) During development, the client authenticates via `POST /api/v1/auth/dev/mobile` with role-based personas (Member, Driver, Admin).
2) The issued opaque token is encrypted and persisted within **Expo SecureStore** (utilizing Android KeyStore and iOS Keychain).
3) Subsequent API invocations attach the header:

$$\text{Authorization: Bearer } \langle \text{token} \rangle \tag{5}$$

4) Logout operations revoke the session on the backend and purge the encrypted token locally.

C. Mobile Screen Hierarchy

The application structure is organized into five primary tab destinations within the `(tabs)` layout group:

D. Native Hardware Device Integration

The mobile application interfaces directly with native device subsystems:

- **Geolocation (`expo-location`):** Retrieves high-accuracy GPS coordinates for nearby pickup matching.
- **Background Execution (`expo-task-manager`):** Tracks driver coordinates while trips are `IN_PROGRESS`, even when the application is minimized.

Table II
MOBILE SCREEN STRUCTURE AND ROUTING MANIFEST

Screen File	Route Path	Core Functionality
`tabs/index.tsx`	`/`	Trip discovery feed & quick search
`tabs/trips.tsx`	`/trips`	Driver trip management
`tabs/inbox.tsx`	`/inbox`	Real-time conversations list
`tabs/profile.tsx`	`/profile`	Dual-role ratings & user profile
`create-trip.tsx`	`/create-trip`	Driver trip publishing workflow
`bookings.tsx`	`/bookings`	Passenger reservations & boarding codes
`requests.tsx`	`/requests`	Pending proposal negotiations
`schedules.tsx`	`/schedules`	Recurring weekly commute setup
`verification.tsx`	`/verification`	Vehicle registration & license submission

- **Push Notifications (`expo-notifications`):** Preconfigured for FCM and APNs push gateways to deliver critical proposal and arrival alerts.
- **Native Frosted Glass (`expo-glass-effect`):** Extends the Urban Mint aesthetic natively to iOS and Android interfaces.

VI. SECURITY, SAFETY, AND PRIVACY ENGINEERING

Dike enforces strict **Defense-in-Depth** principles:

1) **Zero-Trust Client Invariants:** Server-side validation recalculates all business values (prices, distances, seat limits). Client-supplied parameters are treated as untrusted hints.
2) **Location Privacy Protection:** Exact GPS coordinates are never broadcast publicly. The server applies coordinate fuzzing (500m radius obfuscation) to public feeds. Precise coordinates are disclosed only to confirmed passengers and drivers during active journeys.
3) **Confidential Asset Storage:** Sensitive driver licenses and ID cards reside in private storage buckets, accessible strictly via backend-signed, short-lived URLs (60-second TTL).
4) **Multi-Dimensional Rate Limiting:** Redis-backed fixed-window rate limiters defend login, OTP dispatch, friend requests, and messaging against brute-force attacks and resource exhaustion.

VII. TESTING, EXPERIMENTS, AND EVALUATION

A. Automated Testing Pyramid

The platform enforces comprehensive automated verification across all layers:

B. Empirical Test Case Results

Table IV details the empirical validation results across 10 critical operational scenarios:

VIII. LIMITATIONS AND FUTURE DEVELOPMENT ROADMAP

A. Current Limitations

While the codebase demonstrates comprehensive functional completeness, several production-readiness prerequisites remain:

Table III
AUTOMATED TESTING PYRAMID DISTRIBUTION

Testing Layer	Frameworks & Test Suites	Verification Target
Backend Unit	JUnit 5, Mockito, AssertJ	State machines, cost formulas
Architecture	ArchUnit, Spring Modulith	Modular boundaries, no illegal imports
Integration	Testcontainers, MongoDB Replica Set	Distributed transactions, Outbox
Frontend Unit	Vitest, React Testing Library	UI state, validation schemas
Frontend E2E	Playwright (Desktop & Mobile viewports)	End-to-end user journeys
Accessibility	Axe-Core Playwright	WCAG 2.1 AA compliance
Mobile Doctor	Expo Doctor, TypeScript (`tsc`)	Native module and type integrity

- **Commercial SMS Gateway Integration:** The current deployment relies on simulated OTP providers in development; contractual integration with enterprise SMS aggregators (Twilio, Zalo ZNS) is required for production.
- **Automated eKYC Pipeline:** Driver identity approval currently utilizes administrative review workflows; automated optical character recognition (OCR) and facial biometrics require integration.
- **Automated In-App Payments:** The pilot currently records cash/bank transfer mutual acknowledgments; digital wallet gateways (VNPay, MoMo) are deferred pending regulatory review.

B. Future Engineering Roadmap

1) **Machine Learning Matchmaking:** Implementing ML predictive models to analyze habitual commuter trajectories and recommend optimal matches based on temporal habits.
2) **Battery-Optimized Location Batching:** Developing adaptive GPS clustering algorithms on mobile clients to minimize battery consumption during extended highway commutes.
3) **Horizontal Cluster Scaling:** Transitioning containerized services to Kubernetes (K8s) with Redis Cluster and MongoDB Atlas auto-sharding to support scaling beyond 10,000 active users.

IX. CONCLUSION

The **Dike** project successfully delivers an enterprise-grade, secure, and resilient carpooling ecosystem bridging Java 21 Spring Boot Modular Monolith backend services, responsive Next.js 16 web applications, and native React Native Expo SDK 57 mobile clients. By applying advanced architectural patterns (Clean Architecture, Transactional Outbox, Atomic Conditional Writes, and Same-Origin Auth Proxies), the platform demonstrates provable transactional integrity, seamless real-time interactions, and sustainable maintainability. The resulting platform presents an impactful technological solution to regional commuter transport challenges while establishing a rigorous engineering standard for complex enterprise software development.

Table IV
EMPIRICAL VALIDATION OF CRITICAL OPERATIONAL TEST SCENARIOS

Case ID	Scenario Description	Expected System Behavior	Result
TC01	Unauthenticated access to protected route	Clean redirect to `/login`; zero data leakage	PASSED
TC02	Session expiration during active interaction	Client clears stale query cache; prompts graceful re-login	PASSED
TC03	Submission of malformed form payloads	Dual-layer rejection (Zod on client, Jakarta Validation on API)	PASSED
TC04	Dual concurrent acceptance of final remaining seat	Exactly 1 booking succeeds; second receives out-of-seats error	PASSED
TC05	Duplicate submission of identical proposal acceptance	Idempotent response returned; duplicate booking prevented	PASSED
TC06	Unauthorized actor attempts to read private chat	Backend rejects with 403 Forbidden via Security Policy	PASSED
TC07	Driver enters incorrect 4-digit boarding code	Rejection response; lock triggered after 5 consecutive failures	PASSED
TC08	Duplicate cancellation invocations on a single booking	Seat inventory restored exactly once without duplication	PASSED
TC09	WebSocket disconnection and subsequent reconnection	Client issues REST synchronization to recover latest state	PASSED
TC10	Standard member attempts administrative API call	Rejection at Spring Method Security authorization layer	PASSED

ACKNOWLEDGMENT

The student authors express their sincere gratitude to their project advisor, Mr. Huynh Vang (Lecturer, Department of Software Engineering), and FPT University Quy Nhon Campus for their continuous guidance, technical insights, and invaluable support throughout the research and development of this project.

REFERENCES

[1] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.

[2] O. Drotbohm, "Spring Modulith — A deep dive into building modular applications with Spring Boot," VMware Tanzu Documentation, 2024. [Online]. Available: https://spring.io/projects/spring-modulith

[3] C. Richardson, *Microservices Patterns: With examples in Java*, Manning Publications, 2018.

[4] MongoDB Inc., "MongoDB Transactions and Concurrency Control Documentation," 2024. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[5] Vercel, "Next.js App Router Documentation and Architecture Guidelines," 2024. [Online]. Available: https://nextjs.org/docs

[6] Expo, "Expo SDK 57 Documentation and Native Modules Guide," 2024. [Online]. Available: https://docs.expo.dev/

[7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[8] T. T. Nguyen and V. H. Tran, "Carpooling Optimization in Urban Campuses: A Route Corridor Approach," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 5, pp. 4120–4132, 2023.

[9] OpenAPI Initiative, "OpenAPI Specification v3.1.0," 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0

[10] OWASP Foundation, "OWASP Top 10 API Security Risks," 2023. [Online]. Available: https://owasp.org/www-project-api-security/